



دانشکده علوم ریاضی

رشته علوم کامپیوتر

گزارش تمرین 3

محاسبه مساحت با روش قاعده ذوزنقه ای

از طریق OpenMP

تهیه کننده:

ماریا صبوری دودران

شماره دانشجویی: 402222001

تیرماه

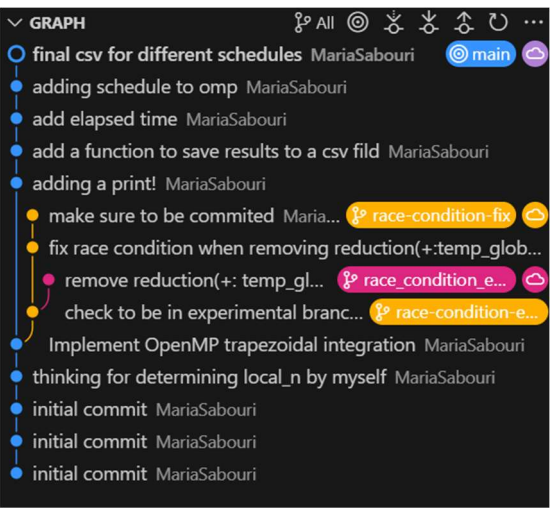
3	 توضیح کد
3	 Main branch
4	 تحلیل روش تقسیم کار
4	 Schedule: static – Chunk-size = Not defined
5	 Schedule: static – Chunk-size = 1
5	 Schedule: dynamic
5	 Schedule: guided
5	 Race-condition-existence branch
6	 Race-condition-fix branch
7	 آزمایش و تحلیل

نکته: لطفا از روی گیت هاب ام چک شود. چون در 3 برنچ من 3 حالت رو بررسی کردم. (در گزارش هر 3 حالت را توضیح داده ام)¹

¹ <https://github.com/MariaSabouri/OpenMP-trapezoidal-integration.git>

توضیح کد

کد در فضای لینکوسی و با زبان c زده شده است. نحوه ی coding من به اینصورت است که در main branch اصلی رو نوشتم. سپس در race-condition-experiment branch اومدن حالتی که به race condition برخورد میکردیم را بررسی کردم. به این منظور، از این branch، branch 2، دیگر هم ساختم که یکی شامل مشکل است و دیگری با اضافه شدن critical section مشکل برطرف شد.²



```
PS F:\Maria\term_8\concurrency\HW3> git branch
* main
  race-condition-experiment
  race-condition-fix
  race_condition_existence
```

Main branch

ابتدا از طریق command-line، تعداد thread ها، schedule type، chunk-size و تعداد تقسیم بندی تابع را دریافت کردم. با تابیین متغیر global_result به عنوان shared variable، تابع Trap را صدا زدم تا عمل اصلی محاسبه مساحت به روش دوزنقه ای را انجام دهد.

² برای بررسی در گیت ها میتوانید از طریق لینک زیر دسترسی پیدا کنید:

<https://github.com/MariaSabouri/OpenMP-trapezoidal-integration.git>

```

int main(int argc, char* argv[]) {
    int thread_count = strtol(argv[1], NULL, 10);
    char* schedule_str = argv[2];
    int schedule = strtol(argv[2], NULL, 10);
    int chunk_size = strtol(argv[3], NULL, 10);
    int n = strtol(argv[4], NULL, 10);

    double global_result = (U(a) + U(b)) / 2;
    time_t start_time, end_time;

    time(&start_time);
    Trap(n, &global_result, thread_count, schedule, chunk_size);
    time(&end_time);

    double elapsed = difftime(end_time, start_time) * 1000000;

    printf("area is: %lf\n", global_result);

    save_results(thread_count, n, global_result, elapsed, EXACT_VALUE, schedule_str, chunk_size);
}

```

در تابع Trap، از طریق تابع های شرطی، روی schedule type های مختلف میتوانم مانور بدم. بخش اصلی کدر در نوشتن pragma است.

```

void Trap(int n,
          double* global_result,
          int thread_count,
          int schedule,
          int chunk_size) {
    double h, x, my_result, temp_global_result;
    int i;

    h = (b - a) / n;
    temp_global_result = *global_result;

    if (schedule == STATIC && chunk_size > 0) {
        #pragma omp parallel for num_threads(thread_count) default(none) \
        reduction(+: temp_global_result) private(i) shared(n, h, chunk_size) schedule(static, chunk_size)
        for (i = 0; i < n; i++) {
            if (i == 0) printf("number of thread(s) is(are): %d\n", omp_get_num_threads());
            temp_global_result += U(a + i*h);
        }
    }
    else if (schedule == STATIC && chunk_size == 0) {
        #pragma omp parallel for num_threads(thread_count) default(none) \
        reduction(+: temp_global_result) private(i) shared(n, h) schedule(static)
        for (i = 0; i < n; i++) {
            if (i == 0) printf("number of thread(s) is(are): %d\n", omp_get_num_threads());
            temp_global_result += U(a + i*h);
        }
    }
}

```

```

}
else if (schedule == DYNAMIC) {
    #pragma omp parallel for num_threads(thread_count) default(none) \
    reduction(+: temp_global_result) private(i) shared(n, h) schedule(dynamic)
    for (i = 0; i < n; i++) {
        if (i == 0) printf("number of thread(s) is(are): %d\n", omp_get_num_threads());
        temp_global_result += U(a + i*h);
    }
}
else {
    #pragma omp parallel for num_threads(thread_count) default(none) \
    reduction(+: temp_global_result) private(i) shared(n, h) schedule(guided)
    for (i = 0; i < n; i++) {
        if (i == 0) printf("number of thread(s) is(are): %d\n", omp_get_num_threads());
        temp_global_result += U(a + i*h);
    }
}

*global_result = temp_global_result;
*global_result *= h;
}

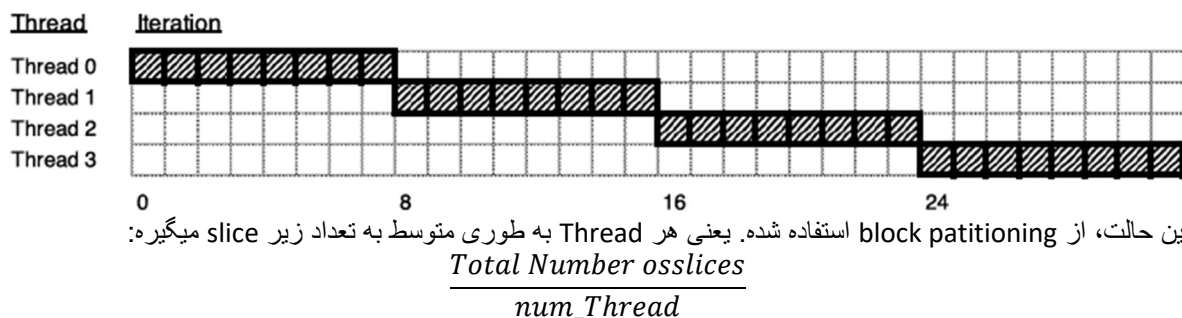
```

به صورت کلی بعد از اینکه thread اصلی (master/parent) به بخش #pragma رسید، توسط parallel به تعداد num_thread ای که مشخص کرده بودم، تقسیم میشه. For این بخش میاد به هر thread، chunk اختصاص میده. توسط default(non) هم برای اینکه خیالم راحت بشه، متغیر های private و share را مشخص کردم. بهش schedule میاد نحوه تخصیص chunk را به هر thread مشخص میکند.

تحلیل روش تقسیم کار

Schedule: static – Chunk-size = Not defined

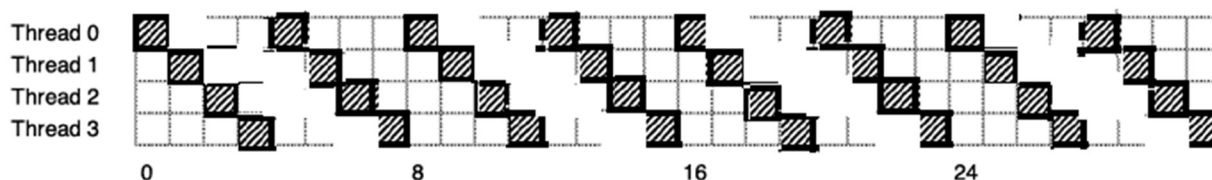
schedule(static)



این حالت برای زمانی خوبه که Load balancing وجود داره. از خوبی هاش میشه به این اشاره کرد که overhead کم تری داره.
Schedule: static – Chunk-size = 1

`schedule(static, 1)`

Thread Iteration

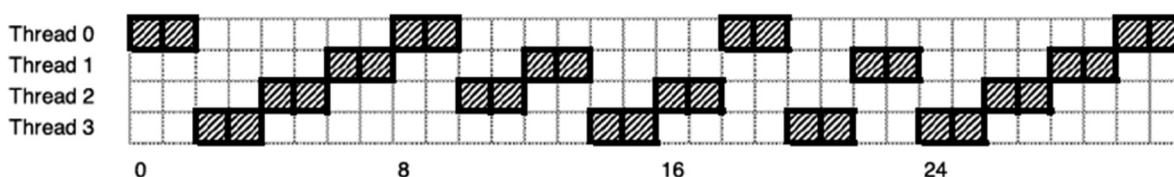


این حالت cyclic partitioning هست. خوبیش اینه که اگر iteration های آخر سنگین تر باشه، Load balancing خوبی انجام شده. به برای cost های نامنظم بهتر از block partitioning هست. ولی بدیش اینه که cash miss بیشتر و Memory locality ضعیف تری داریم.

Schedule: dynamic

`schedule(dynamic, 2)`

Thread Iteration

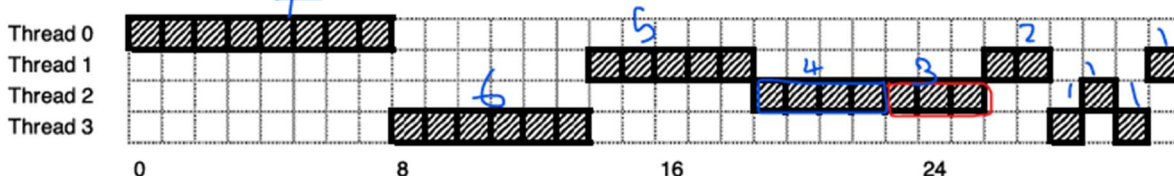


تو این حالت، OpenMP از قبل تصمیم نمیگیره که چه chunk های ثابتی رو هر Thread داشته باش. هر Thread هر وقت Task خودش رو انجام داده، تعداد دیگری iteration میگیره که باز مشغول باشه. این کار باعث میشه که Thread هایی که کارشون رو سریع انجام میدن، بیکار نمونن. بدیش مشکل overheading هست.

Schedule: guided

`schedule(guided)`

Thread Iteration



این حالت مثل حالت قبله ولی با این تفاوت که اول chunk های بزرگی به هر Thread داده میشه و رفته رفته، اندازه chunk های اختصاص داده شده کاهش میابد. Chunk های بزرگ به طور کلی overhead رو کاهش میدن و برعکس، chunk های کوچک load balancing را بهتری میکنن. به نوعی guided تلفیقی از مزیت های static و dynamic هست.

Race-condition-existence branch

در این بخش، هدف این بود که از قصد باعث race condition بشویم. برای این کار، من پارامتر `reduction(+:temp_global_results)` را از کد حذف کردم. این باعث میشد که هیچ محافظتی از متغیر `share` شده نشود.

```

void Trap(int n , double* global_result, int thead_count){
    double h, x, my_result, temp_global_result;
    int i;

    h = (b - a) / n;
    temp_global_result = *global_result;

    #pragma omp parallel for num_threads(thead_count) default(none) private(i) shared(n, h, temp_global_result)

    for (i = 0; i < n; i++) {
        temp_global_result += U(a + i*h);
    }

    *global_result = temp_global_result;
    *global_result *= h;
}

```

این باعث شد که خروجی ناپایداری داشته باشیم.

Num_Threads	Slices	Area Result
1	1000	1.999998
1	1000	1.999998
1	1000	1.999998
2	1000	1.20703
2	1000	1.313468
2	1000	1.227736
4	1000	1.036306
4	1000	1.629917
4	1000	1.583536
8	1000	1.829372
8	1000	0.414524
8	1000	1.598781

Race-condition-fix branch

برای حل مشکل قبل، کافی بود که بخش critical قبل تغییر دادن temp_global_variable بگذارم.

```

void Trap(int n , double* global_result, int thead_count){
    double h, x, my_result, temp_global_result;
    int i;

    h = (b - a) / n;
    temp_global_result = *global_result;

    #pragma omp parallel for num_threads(thead_count) default(none) private(i) shared(n, h, temp_global_result)

    for (i = 0; i < n; i++) {
        #pragma omp critical
        temp_global_result += U(a + i*h);
    }

    *global_result = temp_global_result;
    *global_result *= h;
}

```

همانطور که نتایج جدول زیر نشان میدهد، خروجی جواب ثابت باقی ماند و دچار race-condition نشد.

Num_Threads	Slices	Area Result
1	1000	1.999998
1	1000	1.999998
1	1000	1.999998
2	1000	1.999998
2	1000	1.999998
2	1000	1.999998
4	1000	1.999998
4	1000	1.999998
4	1000	1.999998
8	1000	1.999998
8	1000	1.999998
8	1000	1.999998

آزمایش و تحلیل

همانطور که در جدول زیر نشان داده شده، برای مسئله ما این تعداد خواسته شده، عملاً $1 \cong \text{speedup}$ است. هرچه تعداد Thread ها رو اتفاقاً اگر اضافه کنیم، مقدار Efficiency از مقدار 1 کاهش میابد. یک حالت خاص که دیده شده برای تعداد Thread به مقدار 2، $\text{speedup} = 0$ و $\text{Efficiency} = 0$ شده است. به طور کلی در این مسئله بهتر است که به صورت serial انجام شود.

به طور کلی افزایش تعداد Tread ها باعث Loud محاسباتی بین هسته ها ی Processor ها میشه و بنابراین زمان اجرای برنامه کاهش میابد. با اینحال کاهش زمان اجرا همواره متناسب با تعداد Thread ها نیست. علت این موضوع وجود overhead های ناشی از ساختن و manage کردن Thread ها و race condition ناشی از حافظه مشترک و محدودیت سخت افزاری است. پس با افزایش تعداد Thread ها انتظار داریم که Speedup زیاد بشه به مقدار قابل توجهی ولی پس از رسیدن به تعداد مشخصی از تعداد Thread ها، نرخ رشد کم میشه و حتی افزایش تعداد Thread ها تاثیری بر عملکرد نداشته باشه.

از طرف دیگه، Efficiency حالت موازی، با افزایش تعداد Thread ها کاهش میابد، زیرا بخشی از زمان اجرا صرف عملیات Managing و ارتباط بین Thread ها میشه.

Num_Threads	Slices	Area Result	Elapsed-Time(us)	Exact Area	Schedule	Chunk	Error
1	1000	1.999998	0	2	0	0	2E-06
1	10000	2	0	2	0	0	0
1	100000	2	0	2	0	0	0
1	1000000	2	0	2	0	0	0
2	1000	1.999998	0	2	0	0	2E-06
2	10000	2	0	2	0	0	0
2	100000	2	0	2	0	0	0
2	1000000	2	0	2	0	0	0
4	1000	1.999998	0	2	0	0	2E-06
4	10000	2	0	2	0	0	0
4	100000	2	0	2	0	0	0
4	1000000	2	0	2	0	0	0
8	1000	1.999998	0	2	0	0	2E-06
8	10000	2	0	2	0	0	0
8	100000	2	0	2	0	0	0

8	1000000	2	0	2	0	0	0
1	1000	1.999998	0	2	0	1	2E-06
1	10000	2	0	2	0	1	0
1	100000	2	0	2	0	1	0
1	1000000	2	0	2	0	1	0
2	1000	1.999998	0	2	0	1	2E-06
2	10000	2	0	2	0	1	0
2	100000	2	0	2	0	1	0
2	1000000	2	0	2	0	1	0
4	1000	1.999998	0	2	0	1	2E-06
4	10000	2	0	2	0	1	0
4	100000	2	0	2	0	1	0
4	1000000	2	0	2	0	1	0
8	1000	1.999998	0	2	0	1	2E-06
8	10000	2	0	2	0	1	0
8	100000	2	0	2	0	1	0
8	1000000	2	0	2	0	1	0
8	1000	1.999998	0	2	1	0	2E-06
8	10000	2	0	2	1	0	0
8	100000	2	0	2	1	0	0
8	1000000	2	0	2	1	0	0
4	1000	1.999998	0	2	1	0	2E-06
4	10000	2	0	2	1	0	0
4	100000	2	0	2	1	0	0
4	1000000	2	0	2	1	0	0
4	10000000	2	0	2	1	0	0
2	1000	1.999998	0	2	1	0	2E-06
2	10000	2	0	2	1	0	0
2	100000	2	0	2	1	0	0
2	1000000	2	1000000	2	1	0	0
2	10000000	2	1000000	2	1	0	0
1	1000	1.999998	0	2	1	0	2E-06
1	10000	2	0	2	1	0	0
1	100000	2	0	2	1	0	0
1	1000000	2	0	2	1	0	0
1	1000	1.999998	0	2	2	0	2E-06
1	10000	2	0	2	2	0	0
1	100000	2	0	2	2	0	0
1	1000000	2	0	2	2	0	0
2	1000	1.999998	0	2	2	0	2E-06
2	10000	2	0	2	2	0	0
2	100000	2	0	2	2	0	0
2	1000000	2	0	2	2	0	0
4	1000	1.999998	0	2	2	0	2E-06
4	10000	2	0	2	2	0	0
4	100000	2	0	2	2	0	0
4	1000000	2	0	2	2	0	0
8	1000	1.999998	0	2	2	0	2E-06
8	10000	2	0	2	2	0	0
8	100000	2	0	2	2	0	0
8	1000000	2	0	2	2	0	0

